Hello World 电控组靶车考核说明(骨架工程清单・平衡)

适用骨架:target_skeleton(单板:平衡靶车板)| 2026-10 | 电控组

欢迎加入 Hello World 电控组。这份文档就是你的考核任务书,动手前先通读一遍:

一、**考核概述** —— 我们要考核你什么、安全红线在哪里,务必先读; 二、**机器人与工程结构** —— 这台车和这份骨架长什么样,我们已经给你准备好了哪些代码; 三、**掏空物清单** —— 哪些代码被我们掏空了,要你亲手补上; 四、**任务清单** —— 建议你按依赖顺序完成的活,每一项都是验收项; 五、**验收标准** —— 我们怎么判分、做到什么程度拿什么等级; 六、**附录** —— 整车参数与关键文件索引,写代码时随时回来查。

卡住了先查本文档,再带着你的排查过程来问老队员。上电调试前,重读一遍 1.3 安全红线。

> 图 1:本队靶车实车 —— 两轮平衡 + 旋转靶标单板架构,本次考核的目标就是把这台车的平衡控制补完整。 与步兵(全向轮)考核的双板骨架不同,靶车是**单板车**:一块 STM32F407IGH6 同时负责平衡控制、旋转电机控制与遥控接收,没有板间通信项。

一、考核概述

1.1 目标

以平衡靶车单板骨架(target_skeleton)为底:一块 STM32F407IGH6 独立完成全部工作。我们在骨架里已经给你备好本车用到的**驱动级**组件(M3508 / GM6020 电机驱动、遥控器 DBUS 解码、UART 接收管理、BMI088 + Mahony 姿态解算封装、CMSIS-DSP 等,按 `inc/` + `src/` 平铺给出,只保留本车所需部分;CAN 通信、PID、滤波器等**不提供**)与 CubeMX 底层(1ms 控制中断、IWDG 看门狗),但应用层三层(Instance / RobotModules / Task)全部掏空:没有模式、没有状态机、没有平衡控制、没有闭环。

你要在这份骨架上自己实现:**双轮平衡 LQR 状态反馈控制、旋转电机(GM6020)速度闭环、Dead → Resurrection → Working 状态机与工作模式切换、急停与软件离线保护、任务调度与 IWDG 看门狗喂狗**,使整车能立稳、可控、失效即停,通过功能演示 + GitHub 代码验收。

1.2 范围

项	内容
轮组平衡	2×M3508(两轮)LQR 状态反馈,状态量 [x, v, θ, w],转矩模式闭环
旋转电机	1×GM6020 yaw 速度闭环(电流模式),右摇杆边沿步进调速
工作模式	待机·急停 / 轨道巡航 / 轨道手动 / 手动平衡类,拨杆组合切换
状态机	Dead → Resurrection → Working 三态,上电自检通过后才允许输出
调试保障	遥控器急停、软件离线保护(遥控失联→待机、电机反馈掉线→零输出)、IWDG 看门狗

裁判系统、视觉(NUC)、轨道传感器、图传、键鼠控制、LED 灯效等均不属于本次考核,骨架中也未包含,请勿在这些方向上花时间。

1.3 安全红线(先读,违反直接不通过)

⚠ **本次考核的第一要求:绝不允许任何电机失控持续转动,所有电机动作必须由遥控器指令驱动。** 平衡车失衡倒地时轮子会高速空转,大扭矩底盘电机失控 = 伤人伤车。

红线	说明
禁止电机失控持续转动	不允许上电自转、开环常转、空载常转「测试」,以及任何「没有遥控器指令却一直在动」的持续转动情况;待机与纯平衡模式下旋转电机必须零输出。
必须由遥控器控制	只有收到有效 DBUS 遥控数据(DT7 解码成功),且处于对应档位/模式时,才允许电机输出。
失效即归零	急停档位、生命状态未进入 Working、遥控器失联、电机反馈掉线时,所有电机必须立刻输出归零(`setInput(0)`)并复位控制状态(见 3.6 软件离线保护)。
禁止无约束代码	不允许写无条件循环转动、常值/递增电流等不受遥控器约束的代码。
平衡车调试规程	上电必须先处于待机档再切换模式;调平衡与 LQR 增益时**车轮必须离地架起**;增益只允许小步长修改;禁止把非零参考量写死在初始化里。

演示 / 验收时,一旦发现电机不受遥控器控制或失控持续转动,直接判定不通过。

1.4 提供边界(核心)

我们提供(与兄弟骨架一致,**只给驱动级组件**):**电机驱动**(M3508 / GM6020,纯字节 encode / decode 接口)+ **DT7 遥控解码** + **UART 接收管理**(UartRxMgr,空闲中断 + DMA)+ **IMU 封装**(BMI088 + Mahony,含零漂标定)+ **CMSIS-DSP** + CubeMX 底层(TIM6 1ms 中断、IWDG 已初始化)+ 工程骨架。

我们不提供:**CAN 通信(CAN 收发管理、滤波器、收发热路径)、PID 控制器、滤波器、状态机基类、工作模式、LQR 平衡控制、任务调度、喂狗逻辑**——全部由你自己写。CAN 收发在 `Task/Src/comm_task.cpp` 里用 HAL 自己实现; `Instance/Src/ins_pid.cpp` 与 `ins_filter.cpp` 已把函数签名留好(返回类型为前置声明),你在其中实现自己的类即可。

> 与步兵(全向轮)考核的差异:IMU 封装本次由组件库提供,不需要重写;但 **PID 与速度滤波同样需要你自己实现**;核心考核点转为 **LQR 平衡控制**;单板车没有板间通信项。

维度	我们提供	你要写
CAN 通信	——(不提供)	**自己实现 CAN 收发**:滤波器配置、`HAL_CAN_Start` 、接收中断按 StdId 分发电机 `decode` 、每周期 `encode` 拼帧发送(`comm_task.cpp`)
电机	M3508 / GM6020 驱动类(encode/decode、单位接口)	电机 ID / 输入类型 / 方向 / 减速比等 `OptionalParams` 配置;实例化接线
遥控器	DT7 解码类(18 字节 DBUS → 摇杆 / 拨杆)+ UartRxMgr(空闲 + DMA)	UART 接收注册接线;离线阈值配置

维度	我们提供	你要写
IMU	`Imu` 封装类(BMI088 + Mahony,开机零漂标定)	`Config` 配置(采样数 / kp / 安装旋转矩阵 / SPI 引脚)、`initHardware` 调用时机、零漂收敛判定
控制算法	——(不提供)	**自己实现 PID 与速度滤波器;PID 参数整定;LQR 状态反馈(状态量构建 + 控制律 + 增益);速度参考斜坡;模式逻辑**
状态机	——(不提供)	自己设计状态机基类 / 迁移机制;Dead → Resurrection → Working 三态、工作模式映射、急停最外层
离线保护	`DT7::isOffline()` / `Motor::isOffline()` 超时判定(组件库已实现,约 100ms 阈值)	**把判定接进状态机与指令下发**:遥控失联→待机、电机掉线→零输出
调度	main.c 已启动 TIM6 1ms 中断	MainTaskInit / MainTask、`HAL_TIM_PeriodElapsedCallback` 分发、CommTask
看门狗	IWDG 已初始化(见 6.2 超时参数)	喂狗位置与策略(思考:喂狗应该证明什么还活着?)

1.5 验收形式

功能演示(主要)+ 代码上传 GitHub 验收(参考 2026 考核)。 DDL 与分组规则由组长另行通知。

二、机器人与工程结构

2.0 单板架构与目录布局

整车只有一块 STM32F407IGH6 控制板,同时负责:两轮平衡控制、旋转电机控制、遥控器接收、IMU 姿态解算。没有板间通信。

```
target_skeleton/
├─ Core/                  CubeMX 生成(main.c / can.c / iwdg.c / tim.c / usart.c / spi.c / dma.c /
gpio.c)
├─ Drivers/               STM32F4 HAL + CMSIS
├─ inc/                   ★ 提供的组件头文件(按本车裁剪的公共库,勿改)
├─ src/                   ★ 提供的组件源文件(同上)
├─ third_party/           CMSIS-DSP(ahrs / BMI088 依赖的数学库)
├─ Instance/              实例化 getter(Inc/ 头文件齐全;Src/ 掏空,TODO)
│  ├─ Inc/ ins_all / ins_motor / ins_imu / ins_pid / ins_filter / ins_rc / ins_comm / ins_robot
│  └─ Src/ 同上各自的实现(掏空,TODO)
├─ RobotModules/          ★ 你的任务总纲(掏空,TODO)
│  ├─ Inc/ robot.hpp      顶层状态机(三态 + 模式映射 + 离线保护)
│  └─ Inc/ chassis.hpp    平衡底盘机构(LQR + 旋转电机闭环 + 电机下发兜底)
├─ Task/                  初始化与周期调度(掏空,TODO)
│  ├─ Inc/ main_task.hpp  1ms 控制入口
│  └─ Inc/ comm_task.hpp  通讯层(管理器接线 + 发送冲刷 + HAL 回调)
├─ XiaoKuBao.ioc          CubeMX 工程
├─ CMakeLists.txt / CMakePresets.json    构建配置
└─ target_car.jdebug      SEGGER Ozone 调试工程
```

你不需要动 CMakeLists.txt: `Task` / `Instance` / `RobotModules` 三个目录(`user_folders`)下的 `.c*` 会被自动递归收集(含新建的 Inc / Src 子目录); `inc/` 与 `src/` 下的组件也由构建脚本自动接管。若自建新目录,把目录名加进 CMakeLists.txt 的 `set(user_folders ...)` 即可。

2.1 组件清单(已提供,参考接口勿改)

组件已按本车需要裁剪,头文件在 `inc/` 、源文件在 `src/` (平铺,只给本项目用得到的部分):

组件	内容
电机(`inc/motor*.hpp`)	M3508(C620 电调:发送 0x200 ID1~4 / 0x1FF ID5~8,反馈 0x200+id)与 GM6020(电流模式发送 0x1FE ID1~4,反馈 0x204+id)驱动类; `CreateMotor` 工厂与 `OptionalParams` (方向 / 减速比 / 角度偏移 / 输入类型 kRaw / kTorq / kCurr / 限幅); `angle() / vel() / torq() / curr()` 访问器
遥控器(`inc/DT7.hpp`)	DT7 遥控器:18 字节 DBUS 解码 → `rc_lv / lh / rv / rh` ([-1,1])、 `rc_wheel` 、 `rc_l_switch / rc_r_switch` (kUp / kMid / kDown);接口只做 decode
IMU(`inc/imu.hpp` 、 `inc/BMI088.hpp`)	`Imu` 封装类:BMI088(SPI)+ Mahony 姿态解算;开机静止采样求角速度零漂; `roll() / pitch() / yaw()` 、 `gyro_pitch()` ; `isOffsetCalcFinished()` 收敛判定
通讯(`inc/uart_rx_mgr.hpp` 等)	UartRxMgr(空闲中断 + DMA 接收管理,用于遥控器串口); `Receiver` / `RxMgr` 为接收接口基类。**CAN 收发组件不提供**
姿态解算(`inc/mahony.hpp` 、 `inc/ahrs_base.hpp`)	Mahony 姿态解算(被 `Imu` 封装使用);依赖 `third_party/dsp/CMSIS-DSP`

组件	内容
基础工具(`inc/tick.hpp` 、 `inc/offline_checker.hpp` 等)	`GetTickMs / GetTickUs` 时间基准、离线检测器 `OfflineChecker`(在线超时判定)、内存与断言工具

> ⚠ **不提供的组件**:CAN 收发管理(CanRxMgr / CanTxMgr)、PID 控制器、FIR / IIR / TD 等滤波器、状态机基类 `ModuleFsm` —— 请自己实现:CAN 收发写在 `Task/Src/comm_task.cpp`(HAL 层),算法类新建文件;不要试图在 `inc/` 里找。 `ins_pid.hpp` / `ins_filter.hpp` 只前置声明了 `hello_world::pid::Pid` 与 `hello_world::filter::Td`,等你把类写出来。

2.2 CubeMX 底层现状

项	配置
控制周期	TIM6 已按 1ms 周期配置(预分频 84、重装 1000); `HAL_TIM_PeriodElapsedCallback` 分发是你的任务(见 3.5)
遥控器串口	USART3,100000 baud,9 位数据含偶校验,只收(DBUS)
CAN	CAN1(电机总线)+ CAN2(预留)外设已由 CubeMX 初始化;滤波器配置、 `HAL_CAN_Start` 、收发中断与报文分发**由你自己实现**(骨架不提供 CAN 通信组件)
IMU	SPI1,BMI088(加速度 CS = PA4,陀螺仪 CS = PB0)
IWDG 看门狗	已初始化并启动计数:预分频 16 / 重装 500 → **约 250ms 超时**。全工程目前没有任何喂狗代码(见下方提示)

⚠ **注意:骨架上电后若不喂狗,板子会在超时时间后复位并无限循环(约 0.25s 一轮)。这是预期行为,不是板子坏了** —— 请尽快实现喂狗,并想清楚「喂狗应该证明什么还活着」(见 3.5 与 4.9)。

> 一个常见但**错误**的喂狗位置:在「遥控器串口收到数据」的回调里喂狗。这样遥控器失联 → 板子 250ms 后复位 → 复位后 1s 启动延时内主循环根本没跑 → 电机收不到任何零指令帧 → 再复位……整机进入复位循环,而"停车"完全依赖电调自身的指令超时。正确做法是:**看门狗由 1ms 控制中断喂(证明控制循环活着),遥控器失联由软件离线保护处理(见 3.6)。**

2.3 电机协议速查

M3508(C620 电调,左右两轮):控制帧 0x200(ID 1~4),每帧 4 台电机 × int16 大端;反馈帧 ID = 0x200 + id。

参数	M3508(C620)
报文输入范围	±16384
最大电流	20 A
内置减速比	3591 / 187 ≈ 19.2
本车外置减速比	268 / 17 ≈ 15.76(支撑轮传动,配 `ex_redu_rat` 时填外置值)

参数	M3508(C620)
角度分辨率	2π / 8191 rad

GM6020(旋转电机):电流模式控制帧 0x1FE(ID 1~4),反馈帧 ID = 0x204 + id;反馈速度噪声较大,建议自行实现滤波后用于速度环。

三、掏空物清单(这些都要你自己写)

3.1 Instance 层(Instance/Src/ 全部掏空;Inc/ 头文件已给全)

文件	要写什么
`ins_motor.cpp`	静态定义左 / 右 M3508 与旋转 GM6020 对象并返回指针。`OptionalParams`:轮组 `kTorq` + 外置减速比 268/17 + 角度范围 kNegInfToPosInf;旋转 `kCurr` + 方向按实机
`ins_imu.cpp`	填写 `Imu::Config`(零漂采样数、Mahony kp、安装旋转矩阵、SPI 引脚)并静态构造。⚠ 不要在 getter 里重复构造——每调一次都会重置 Mahony 状态与零漂标定
`ins_pid.cpp`	**自己实现 PID 类**(位置式 / 增量式,含 Kp / Ki / Kd、积分限幅、输出限幅),放在自己新建的文件里(命名空间 `hello_world::pid`,类名 `Pid` 或改返回类型),再在这里静态定义旋转电机速度环对象并整定参数(输出限幅 ±3A 量级)
`ins_filter.cpp`	**自己实现速度滤波器**(一阶低通 / TD 均可),放在自己新建的文件里(命名空间 `hello_world::filter`,类名 `Td` 或改返回类型),再在这里静态定义并使用;不使用时可返回 `nullptr`
`ins_comm.cpp` / `ins_rc.cpp`	UartRxMgr / DT7 的静态对象与 getter;DT7 离线阈值(建议 100ms)。CAN 层不设 Instance 对象,直接在 `comm_task.cpp` 实现
`ins_robot.cpp`	**依赖接线的唯一入口**:静态定义 Chassis 与 Robot、完成全部 register 注册后返回 Robot 指针(做法见 3.2)

3.2 RobotModules(robot.* / chassis.*,掏空核心)

本车按「**编排 + 机构**」拆成两个类(命名空间 `robot`),这也是战队其余项目的通用结构:

`robot::Robot`(顶层状态机,状态机基类 / 迁移机制自己设计)

> 原工程用组件库的 `hello_world::module::ModuleFsm` 三态基类,**本骨架不提供。**你可以设计自己的基类或直接在 `Robot` 里写迁移函数;可以参考 `code_control/` 下其他项目的 `Robot` 实现,但不要直接拷贝。

- **状态机**:Dead → Resurrection → Working 三态。Dead 输出全零;Resurrection 等待 IMU 零漂收敛 + 全部电机反馈在线 + 上电延时(1s 量级)后才进 Working;**急停判断放最外层**(待机档 → 所有电机指令立即归零,硬性验

收项);状态 / 模式切换时复位残留指令与积分状态,防电机突变。

- **工作模式**:按左 / 右拨杆组合映射(见 3.4 模式表),把过死区后的摇杆量与目标模式下发给底盘机构。
- **离线保护**:遥控器 `isOffline()` 时强制待机(见 3.6)。
- 只做编排,不直接操作电机。

`robot::Chassis` **(平衡底盘机构)**

- **轮组 LQR 平衡**:见 3.3 硬性约定。
- **旋转电机闭环**:见 3.4。
- **指令下发与掉线兜底**:每周期把指令写到电机并标记发送;电机反馈掉线时零输出 + 复位 PID(见 3.6)。
- 电机、IMU、PID 通过 `registerXxx()` 注册进来(空构造 + 注册,不自己 `Create*()`)。

依赖接线全部收在 `Instance/Src/ins_robot.cpp` 的 `CreateRobot()` 里(静态单例 + 首次调用注册),`main_task` 只调用 `CreateRobot()` ——不要在 `main_task` 里逐个 `Create*()` 再手工注册。

3.3 LQR 硬性约定:状态向量与符号(先在纸上推,再写代码)

⚠ **本车的核心控制是轮式倒立摆的 LQR 状态反馈。状态向量与符号约定全工程必须一致——符号错了,车会「加速倒下」而不是平衡。** 写代码前先在纸上画图(车体坐标系、正 pitch 方向、左右轮正转向),推导每一项的符号,再落地;符号以实车架起调试为准。

- 状态向量 **[x, v, θ, w]**:x = 轮组位移(m,由 v 以 1ms 周期积分)、v = 左右轮速度均值(**左轮状态符号 +1、右轮 −1**)、θ = 车体 pitch(`imu->pitch()`)、w = pitch 角速度(`imu->gyro_pitch()`)。
- 控制律:**u = −(k_x•err_x + k_v•err_v + k_θ•err_θ + k_w•err_w)**,err = 反馈 − 参考。
- 输出符号:左轮输出 **+u**、右轮输出 **−u**;**转向量 turn 不进入 LQR 状态向量,单独叠加**:left = +u + turn,right = −u + turn(转向由左摇杆横向直接给出并限幅)。
- 参考量:θ_ref = 0、w_ref = 0;v_ref 来自遥控器(**必须经斜坡限幅**,建议 2 ref/s 量级,防指令突变翻车);x_ref 由 v_ref 积分;进入位置保持类模式时把 x_ref 锁定为当前 x_fdb,避免切模式瞬间位置误差暴冲。
- 增益 k_x / k_v / k_θ / k_w 待你整定(附录 6.3 给一组本车参考值);建议先建倒立摆模型离线求解 K 阵(加分项),实车从架起状态小步长调。
- **待机模式下必须把 LQR 全部参考 / 反馈 / 输出状态清零**,防止残留状态导致切模式瞬间暴冲。

3.4 工作模式与旋转电机(验收按此执行)

模式	触发(左 / 右拨杆)	行为定义
待机·急停 Standby	左下,或左上+右下,或未定义组合	所有电机强制零输出;LQR、PID、参考量全部清零复位。**上电默认态**
轨道巡航 RailCurise	左上 + 右上	轮组速度随左摇杆纵向(进入时 x_ref 锁定当前位移);旋转电机闭环工作。(轨道闭环逻辑预留)
轨道手动 RailManual	左上 + 右中	轮组锁定当前位置零输出,仅旋转电机闭环。(轨道逻辑预留)
手动平衡 ManualLocked / Balance / ManualSpin	左中 + 右上 / 右中 / 右下	轮组 LQR 平衡:左摇杆纵向控速度参考、横向给转向叠加;旋转电机闭环。三档当前共用行为,预留差异扩展

旋转电机操控约定:右摇杆横向(`rc_rh`)**边沿触发**步进调速 —— 推过阈值(0.8)一次,速度参考 ±4 rad/s 步进,限幅 ±35 rad/s;经**自实现的 PID**(参考 kp 量级 0.05、输出限幅 ±3A)+ 自实现的速度反馈滤波闭环。待机与纯平衡模式下旋转电机必须保持零输出并复位 PID。

3.5 Task 层(main_task.cpp / comm_task.cpp)

Task 层只做「接线 + 按周期调用」,不写业务逻辑;按职责拆成两个文件:

文件 / 位置	要求
`main_task.cpp` · MainTaskInit	按依赖顺序: `robot_ptr = CreateRobot()` (接线已在工厂内完成)→ `CreateImu()->initHardware()` (阻塞,勿放中断)→ `CommTaskInit()` → `HAL_TIM_Base_Start_IT(&htim6)`。在 main.c 的 USER CODE 区 while(1) 之前调用
`main_task.cpp` · MainTask	1ms 周期入口,保持薄薄一层:只调 `robot_ptr->update()` + `robot_ptr->run()`
`main_task.cpp` · 定时器分发	`HAL_TIM_PeriodElapsedCallback`:先判断中断来自 htim6 → **喂 IWDG(在启动延时门控之前)** → 启动延时(1s,等外设稳定)后调 `MainTask()` + `CommTask()`
`comm_task.cpp` · CommTaskInit	**自己实现 CAN**:滤波器配置 + `HAL_CAN_ConfigFilter` + `HAL_CAN_Start(&hcan1)` + 打开 RxFifo0 中断;UART:拿 UartRxMgr → `addReceiver` (DT7)→ `startReceive()`
`comm_task.cpp` · CommTask	每 1ms 遍历三台电机: `encode(len, data)` 后按 `txId()` 发帧;左 / 右 M3508 的发送 ID 相同,必须合并成一帧(8 字节按 id 排位)
`comm_task.cpp` · CAN 回调	`HAL_CAN_RxFifo0MsgPendingCallback` 已给出取帧框架,你按 `StdId` 与各电机 `rxId()` 比对后调 `decode(8, data, StdId)`,并重新使能中断
`comm_task.cpp` · UART 回调	`HAL_UARTEx_RxEventCallback` (huart3)→ UartRxMgr; `HAL_UART_ErrorCallback` → UartRxMgr。**不要在这里喂狗**,原因见 2.2

3.6 软件离线保护(硬性验收项)

组件库里 `DT7::isOffline()` 与 `Motor::isOffline()` 都已实现(默认阈值 100ms,分别在解码 / 反馈时喂),但**必须由应用层主动查询才有意义**——不查,遥控器拔掉后摇杆值会永远冻结在最后一帧上;如果失联时恰好在「轨道巡航」档,车会带着最后一次的速度参考持续跑飞。

要做两层保护:

1. **遥控器失联 → 强制待机**(`robot::Robot::updateWorkingMode()` 入口):

```
if (rc_ptr_ == nullptr || rc_ptr_->isOffline()) {
  working_mode_ = Chassis::WorkingMode::kStandby;
  return;
}
```

走 Standby 即自然获得「三电机零输出 + LQR/PID/参考量清零」;遥控器恢复后自动回到正常的拨杆映射,无需复位。

2. **电机反馈掉线 → 零输出兜底**(`robot::Chassis::applyCmd()` 下发处):
   - 下发前判断 `motor->isOffline()` ;为真则 `setInput(0)` (旋转电机再 `pid->reset()`),但**仍然下发这一帧零指令并** `setNeedToTransmit()` ——掉线恢复的第一帧必须是零,不能把掉线期间攒下的指令打出去;
   - 轮组 LQR 在任一驱动轮反馈掉线时暂停 `v_fdb / x_fdb` 积分并清空 LQR 状态,避免复联瞬间用陈旧积分暴冲;
   - 状态机 Resurrection → Working 的门控里加 `chassis->isAllMotorsOnline()` ,电机没起来不进入工作态。

验收:运行中关闭遥控器 → 车在 100ms 阈值内归零停下,不跑飞、不复位;恢复遥控器后可继续操作。拔掉任一电机反馈线(或断 CAN)→ 对应输出归零,重新接回不出现突跳。

四、你的任务清单(验收项,按依赖顺序)

1. **CAN 收发 + 电机实例化** —— ins_motor 配好 ID / 输入类型 / 方向 / 减速比;自己实现 CAN 滤波器 / 发送 / 接收分发(comm_task),跑通「指令下发 + 反馈解析」。
2. **UART DBUS 接收** —— UartRxMgr 注册 DT7,空闲 + DMA 收 18 字节解码,摇杆 / 拨杆数据可用。
3. **IMU 配置与标定** —— Config 填写、 `initHardware` 时机正确;静止采样零漂收敛判定,**未收敛不得进 Working**。
4. **状态机三态** —— Dead / Resurrection / Working 迁移条件与残留指令复位;上电默认待机。
5. **旋转电机速度闭环** —— 自己实现 PID 与速度滤波、PID 参数整定、边沿步进逻辑;空载阶跃响应稳、无持续振荡。
6. **轮组 LQR 平衡(核心)** —— 状态量构建、控制律实现、增益整定;架起验证输出方向 → 落地立稳。
7. **工作模式映射** —— 拨杆组合 → 模式,行为符合 3.4 表;模式切换无突跳。
8. **急停逻辑(硬性验收项)** —— 待机档所有电机归零 + 状态清零;演示:运行中打急停档,车立即停。
9. **遥控器失联保护(硬性验收项)** —— `DT7::isOffline()` 为真时强制待机(见 3.6);演示:运行中关闭遥控器,车在 100ms 量级内归零停下,**且不复位、不跑飞**;恢复遥控后可继续操作。
10. **电机反馈掉线保护(硬性验收项)** —— 下发处 `Motor::isOffline()` 兜底,掉线帧零输出并复位 PID;演示:断任一电机反馈线,对应输出归零,接回不突跳。
11. **IWDG 看门狗(硬性验收项)** —— 在 **1ms 控制中断(htim6 分支)且启动延时门控之前**周期喂狗,证明「控制循环还活着」;验收时临时屏蔽喂狗(或制造任务卡死),板子必须在超时时间内复位(约 0.25s)。注意:**不能**在遥控器串口接收回调里喂狗(原因见 2.2),遥控器失联由第 9 项软件保护处理。
12. **联调演示** —— 架起调平衡 → 落地静止立稳 → 低速直线 / 转向 → 模式切换 → 急停 → 关遥控器停车。

⚠ 以上全部任务都必须遵守 1.3 安全红线:电机不得失控持续转动、必须由遥控器控制、失效即归零;违反直接不通过。

加分项:倒立摆建模推导与 K 阵离线求解文档;LQR 增益整定记录文档;旋转电机由速度环改位置环;上位机 / 调试接口辅助调参;RailCurise 真轨道逻辑;开机自检指示。

五、验收标准

等级	要求
通过	完成 1~12 全部基础项:车落地立稳 ≥30s;遥控操纵正常;急停归零;关遥控器后 100ms 量级内零输出停车(软件保护);断电机反馈零输出;看门狗屏蔽演示复位成功
良好	通过基础上:模块封装清晰(电机 / PID / IMU / 模块各自成类)、代码规范、Git 提交历史合理;模式切换平顺无突跳;平衡受扰(轻推)可恢复、无超调震荡
优秀	完成加分项之一及以上,并能讲清设计取舍(如倒立摆建模与 K 阵求解方法)

⚠ **一票否决项**:出现电机失控持续转动、电机不受遥控器控制、急停不归零、遥控器失联后仍持续输出(未做离线保护)、任务卡死看门狗不复位(谎称已实现)、抄袭,直接判定不通过。

六、附录:关键文件索引与机器人信息

6.1 关键文件索引

先读: `RobotModules/Inc/robot.hpp` 与 `chassis.hpp` (你的任务总纲)、 `Task/Src/{main_task,comm_task}.cpp` 、`Instance/Inc/` 下全部 `ins_*.hpp` (接口已给全)。 **要写的文件**: `Instance/Src/*.cpp` (七个 ins_*,含 `ins_robot.cpp` 接线)、 `RobotModules/{Inc,Src}/{robot,chassis}.{hpp,cpp}` 、`Task/Src/{main_task,comm_task}.cpp` 。 **参考接口(勿改)**: `inc/` + `src/` (按本车裁剪的**驱动级**组件:电机、遥控器、IMU、UART 接收管理、Mahony、基础工具)。⚠ **CAN 通信**、PID、滤波器、状态机基类**不在其中**:CAN 层写在 `comm_task.cpp` ,算法类新建文件实现。 **底层(CubeMX 生成,不要手改)**: `Core/Src/{main,can,iwdg,tim,usart,spi,dma,gpio}.c` ;USER CODE 区内可加代码。

6.2 机器人信息(平衡靶车)

主控与外设

项	配置
MCU	STM32F407IGH6 ×1,C++20,控制周期 1ms(TIM6)
CAN	CAN1(电机总线);CAN2 预留
DBUS(DT7)	USART3,100 kbps,9bit 含偶校验,18 字节,只收;空闲 + DMA 接收
IMU	BMI088(SPI1,acc CS = PA4 / gyro CS = PB0)+ Mahony
IWDG	预分频 16、重装 500 → 约 250ms

电机配置(ID / 方向以实机接线为准,待填处发放前统一补齐)

电机	数量	类型	CAN 反馈 ID	输入类型	方向 / 偏移	说明
左轮	1	M3508	0x200 + id	kTorq(转矩)	待填	外置减速比 268/17

电机	数量	类型	CAN 反馈 ID	输入类型	方向 / 偏移	说明
右轮	1	M3508	0x200 + id	kTorq(转矩)	待填	外置减速比 268/17
旋转 yaw	1	GM6020	0x204 + id	kCurr(电流)	待填	速度闭环 + 速度滤波

遥控器映射(DT7)

输入	用途
左拨杆 + 右拨杆组合	工作模式切换(见 3.4 表)
左摇杆纵向 rc_lv	轮组速度参考(死区 0.05)
左摇杆横向 rc_lh	转向叠加(不进 LQR 状态量)
右摇杆横向 rc_rh	旋转电机步进调速(阈值 0.8,±4 rad/s 步进,限幅 ±35 rad/s)

6.3 LQR 参考增益(本车一组整定值,以实车为准)

增益	参考值
k_x	−10.0
k_v	−27.5435
k_θ	−130.5105
k_w	−12.2422

注:增益与机体参数(重心高度、质量、轮距)强相关,换车必重整定;参考值仅给量级与符号,不保证直接可用。

6.4 构建与烧录

项	方法
构建	`cmake --preset Debug` 后 `cmake --build --preset Debug` (需 arm-none-eabi-gcc 在 PATH;产物 `build/Debug/target-car.elf / .hex / .bin`)
烧录 / 调试	SEGGER Ozone 打开 `target_car.jdebug` (J-Link,SWD);注意该工程指向 `build/target-car.elf` ,构建后需从 `build/Debug/` 拷贝

安全总则(重复一遍)

禁止任何电机失控持续转动;所有电机动作必须由遥控器控制(有效 DBUS 数据 + 对应档位);急停档 / 未进 Working:一律 `setInput(0)` 归零;上电调试时车轮必须离地;首次整机测试前先单板单电机验证方向;增益小步长修改,禁止把非零参考写死在初始化里。